A Neural Network for Authorship Stylometry

Given a passage written by one of K known authors, can we recover who wrote it from writing style alone? Where K-means asked whether style clusters*, here we ask a supervised question: train a multi-layer perceptron on labelled passages and let it learn the decision boundaries between authors directly.*

1 Classifying Five Authors with an MLP

Dataset and preprocessing. We use five LessWrong authors — `Eliezer Yudkowsky`, `John Wentworth`, `Raemon`, `Scott Alexander`, and `Zvi` — with roughly 150 passages each, for **723 labelled passages** in total. From each passage we extract the same **107 stylometric features** used throughout this project, spanning seven groups: word/sentence-length statistics, vocabulary richness (type-token ratio, hapax ratio, Yule K, Simpson D, Brunét W, Honoré R), function-word frequencies, stylistic word-category rates, punctuation rates, coarse POS-tag distributions, and character-level ratios. None of these depend on *what* a passage is about, only on *how* it is written.

Every feature is z-scored with `sklearn`'s `StandardScaler` (mean 0, standard deviation 1) before training. Crucially, the scaler is fit on the training split only and then applied to the dev and test splits, so no test-set statistics leak into training.

The classifier. A multi-layer perceptron (MLP) maps a feature vector $\mathbf{x} \in \mathbb{R}^{107}$ to a probability over the K authors. Each hidden layer applies an affine transform followed by the **ReLU** nonlinearity,

$$\mathbf{h}^{(\ell)} = \mathrm{ReLU}(W^{(\ell)}\mathbf{h}^{(\ell-1)} + \mathbf{b}^{(\ell)}), \qquad \mathrm{ReLU}(z) = \max(0, z),$$

with $\mathbf{h}^{(0)} = \mathbf{x}$. The final layer produces K logits that a **softmax** turns into class probabilities,

$$p_k(\mathbf{x}) = \frac{\exp(z_k)}{\sum_{j=1}^{K} \exp(z_j)},$$

and training minimises the multi-class **cross-entropy** loss over the training passages,

$$L = -\frac{1}{N} \sum_{i=1}^{N} \sum_{k=1}^{K} \mathbf{1}[y_i = k] \, \log p_k(\mathbf{x}_i).$$

Weights are learned by backpropagation with the **Adam** optimiser (`sklearn.neural_network.MLPClassifier`, `solver="adam"`). All hidden layers use 64 units; we vary only the *number* of such layers.

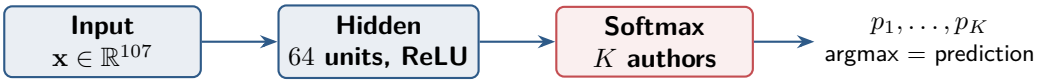


Figure 1: The winning architecture: a single hidden layer of 64 ReLU units feeding a K-way softmax. Deeper variants stack more 64-unit layers between input and output (Section 3).

Training protocol. We split the 723 passages **60/20/20** (train / dev / test), stratified by author, and use the dev split to choose a model before ever touching the test split. Specifically we grid-search over three feature subsets (the top 30, top 50, and all 107 features, ranked by a univariate filter) crossed with four network depths, training each combination on the train split and scoring it on dev. The winner is then **retrained on train+dev combined** and evaluated *once* on the held-out test set.

With only ∼430 training passages and thousands of weights, an MLP can memorise the training set. We pass `early_stopping=True`: `sklearn` holds out 10% of the training data as an internal validation set and halts when its score fails to improve for `n_iter_no_change=15` consecutive epochs (`batch_size=32`, `max_iter=500`). This curbs over-fitting and makes the depth comparison fair — every architecture is trained under the same stopping rule.

Model selection on the dev set. Figure 2 shows dev accuracy for every feature-subset × depth combination. The clear winner is **all 107 features with a single hidden layer** (dev accuracy 0.903). Two patterns already stand out: using all 107 features never hurts, and the deepest network (50 layers) collapses to chance — a phenomenon we return to in Section 3.

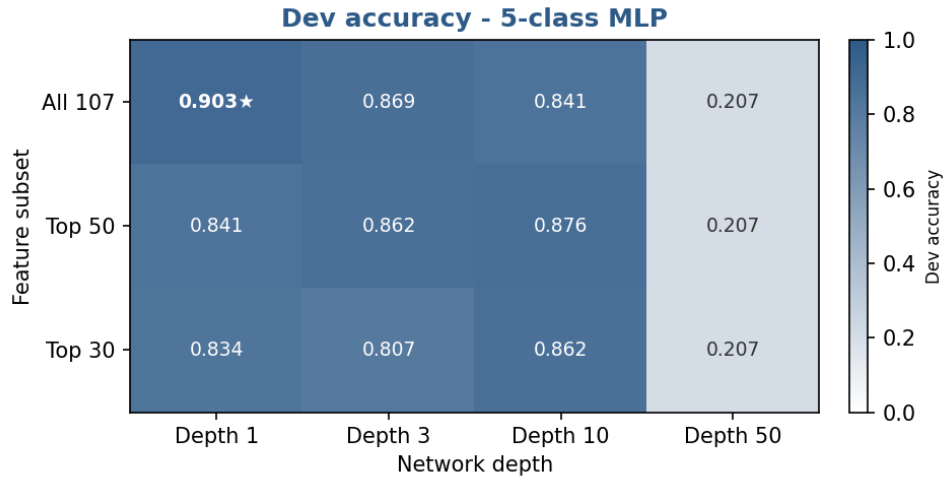


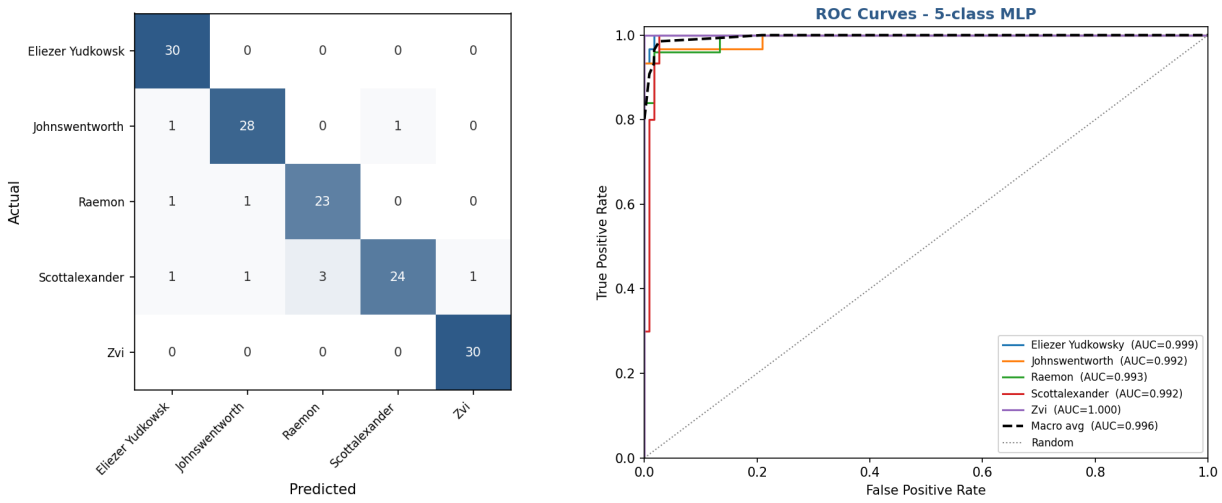
Figure 2: Dev accuracy for the 5-class MLP across feature subsets (rows) and network depths (columns). The starred cell — all 107 features, depth 1 — is the selected model. Depth 50 sits at 0.207, barely above the five-class random baseline of 0.20.

Results. Retrained on train+dev (578 passages) and evaluated on the 145 test passages, the selected model attains the scores in Table 1.

Table 1: Test-set performance of the selected 5-class MLP (all 107 features, depth 1).

Metric	Value
Accuracy	0.931
Weighted F1	0.930
ROC-AUC (macro OvR)	0.995

The confusion matrix (Figure 3a) shows a strong diagonal. `Zvi` and `Eliezer Yudkowsky` are recovered almost perfectly (recall 1.00), while the few errors concentrate on `Scott Alexander` (recall 0.80), whose passages are occasionally mistaken for `Raemon`. The ROC curves (Figure 3b) sit in the top-left corner for every author, with a macro-averaged AUC of 0.995: the model's ranking of "how likely is this author" is near-perfect even where its hard `argmax` decision slips.



(a) Confusion matrix (rows = actual, columns = predicted). **(b)** One-vs-rest ROC curves with macro average.

Figure 3: Test-set diagnostics for the 5-class MLP.

2 Open-Set Recognition: the "None of the Above" Problem

The 5-class model assumes every passage was written by one of the five known authors. In the real world that assumption fails: a passage may come from a *sixth*, unknown author, and a closed-set classifier will confidently mislabel it as whichever of the five it resembles most. Can the network learn to say "none of these"?

Constructing a "none" class. We add a sixth class, `none_of_the_5_authors`, built from **15 other LessWrong authors** not among the five (`ricraz`, `benquo`, `abramdemski`, `sarahconstantin`, `holdenkarnofsky`, `gordon-seidoh-worley`, `screwtape`, `buck`, `turntrout`, `nunosempere`, `benito`, `petermccluskey`, `joe-carlsmith`, `adamshimi`, `tsvibt`), sampling 10 articles from each for **150 "none" passages**. Merged with the five known authors this gives **873 passages over 6 classes**. The training protocol is otherwise identical to Section 1.

Results. Dev-set model selection again picks **all 107 features with a single hidden layer** (dev accuracy 0.943; Figure 4). Retrained on train+dev (698 passages) and evaluated on 175 test passages, it scores as shown in Table 2 — *higher* than the closed-set 5-class model on every metric.

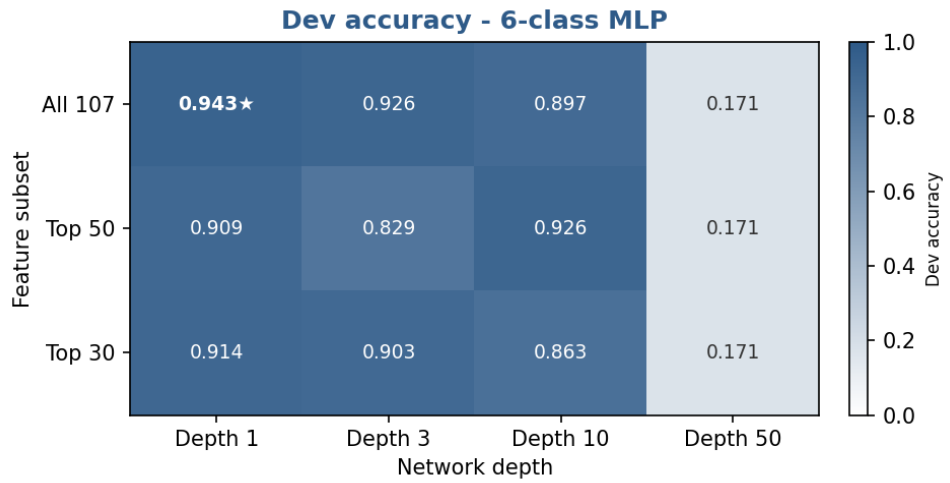
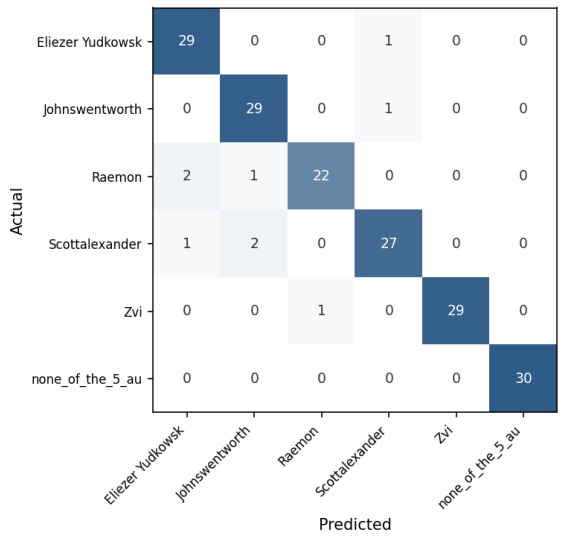


Figure 4: Dev accuracy for the 6-class (open-set) MLP. The selected model is again all 107 features at depth 1, and depth 50 again collapses to the six-class random baseline of 0.167.

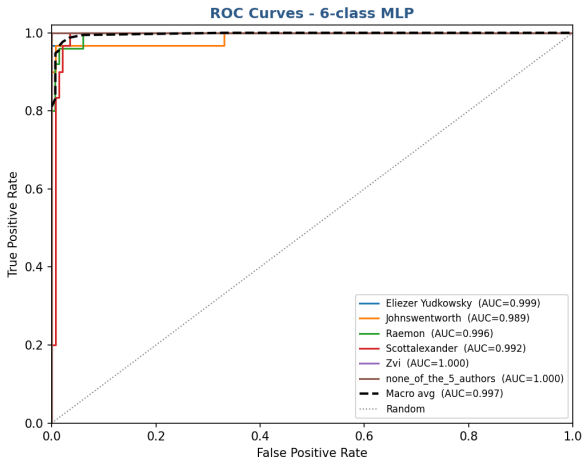
Table 2: Test-set performance of the selected 6-class MLP (all 107 features, depth 1).

Metric	Value
Accuracy	0.949
Weighted F1	0.949
ROC-AUC (macro OvR)	0.996

The striking result is the `none_of_the_5_authors` class itself: **precision, recall, and F1 are all** 1.00 (Figure 5a). Not a single unknown-author passage is misattributed to one of the five, and not a single known-author passage is dumped into the "none" bin. The 15 unknown authors, pooled together, occupy a region of style-space that is cleanly separable from the five targets.



(a) Confusion matrix. The `none` row and column are perfectly isolated.



(b) One-vs-rest ROC curves with macro average.

Figure 5: Test-set diagnostics for the 6-class open-set MLP.

> Why add a class instead of simply flagging low-confidence predictions from the 5-class model as "unknown"? We tried that indirect route, thresholding the 5-class softmax probabilities, and it failed completely: unknown passages routinely produced *high*-confidence predictions for one of the five authors, so a confidence threshold caught essentially none of them. Giving the network labelled "none" examples and letting it learn the boundary directly is far more effective — here, perfectly so.

Why does adding a class *help*? Counter-intuitively, the 6-class model out-performs the 5-class one (0.949 vs. 0.931 accuracy). Two factors contribute. First, the perfectly separable `none` class adds 30 easy test passages, lifting the average. Second, and more interestingly, the contrast against 15 stylistically diverse "other" authors appears to sharpen the boundaries among the five targets rather than blur them: the model learns not just what each target author looks like, but what they do *not* look like.

3 How Deep Should the Network Be?

A natural instinct is that deeper networks are more powerful, so they should classify better. Our grid search says otherwise. Figure 6 plots dev accuracy against depth for each feature subset, on both the 5-class task of Section 1 and the 6-class task of Section 2.

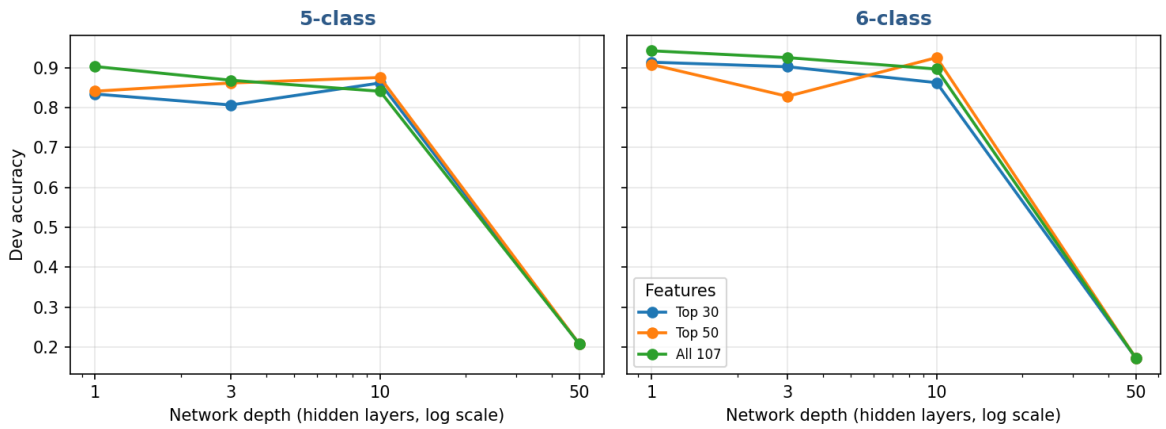


Figure 6: Dev accuracy vs. network depth (number of 64-unit hidden layers, log scale). Performance is flat-to-slightly-declining from depth 1 to depth 10, then **collapses to chance at depth 50** on both tasks.

From depth 1 to depth 10 accuracy barely moves — the extra layers buy nothing because the decision boundaries between these authors are already well within reach of a shallow network on 107 informative features. At depth 50 every configuration collapses: dev accuracy falls to 0.207 (5-class) and 0.171 (6-class), essentially the random baselines of $1/5 = 0.20$ and $1/6 = 0.167$. The network has stopped learning anything.

This is not over-fitting — an over-fit network would still score well above chance on dev. It is an *optimisation* failure. Stacking 50 plain fully-connected ReLU layers with no residual connections or normalisation produces vanishing gradients: the error signal is attenuated layer by layer during backpropagation, so the early layers receive almost no useful gradient. Combined with early stopping on a small dataset, training halts before the network escapes its near-random initialisation. The lesson is concrete: for tabular stylometric features, **a shallow MLP is not just sufficient but preferable**.

Are the gradients actually vanishing? The vanishing-gradient explanation is testable, so we test it. We re-implement the same architecture in numpy — Glorot-uniform initialisation with bound $\sqrt{6/(\text{fan\_in} + \text{fan\_out})}$ (exactly what `sklearn` uses for ReLU layers), ReLU hidden units, softmax output, cross-entropy loss — and read off the gradient of the loss with respect to every layer's weights from a single forward/backward pass on the standardised 5-class features. Averaging over 50 random initialisations, Figure 7 reports the root-mean-square (RMS) of the weight gradients at each layer, at the moment training begins.

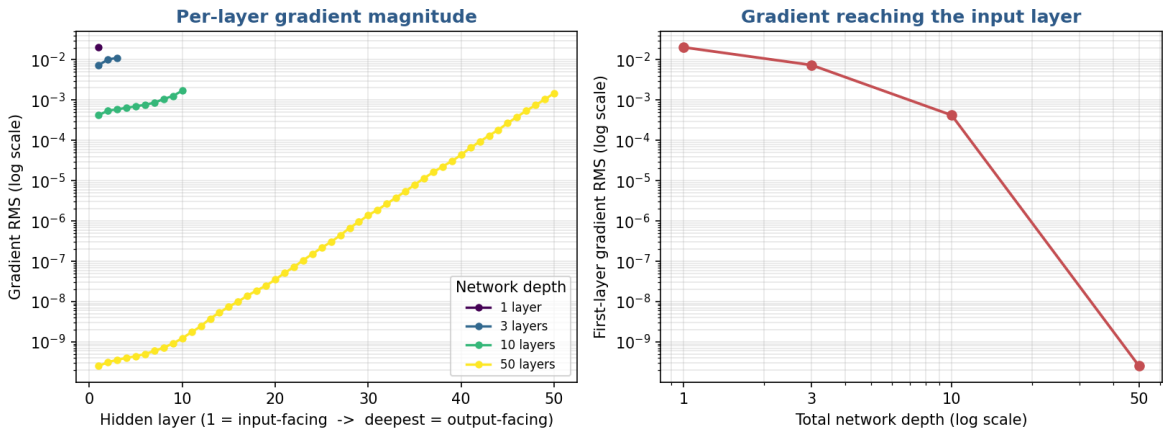


Figure 7: Gradient health at initialisation. *Left*: RMS weight gradient per layer, from the input-facing layer (1) to the output-facing layer, one curve per depth (log scale). *Right*: the gradient reaching the input-facing first layer as a function of total depth. Both axes are logarithmic.

The signature is unmistakable. In the shallow networks the gradient is roughly uniform across layers — at depth 1 the input layer already sees an RMS gradient of 2.0×10^{-2}, only $3.5\times$ smaller than the output layer. As depth grows the gradient decays geometrically on its way back to the input, so the first layer is starved (Table 3). By 50 layers the input-facing layer receives an RMS gradient of 2.5×10^{-10} — eight orders of magnitude weaker than in the shallow net, and 2×10^{7} times weaker than the output layer of the *same* network.

Table 3: RMS weight gradient at initialisation, averaged over 50 inits. The first layer (input-facing) is the one that must learn feature combinations; its gradient collapses with depth.

Depth	First-layer grad	Output grad	Ratio (out / first)
1	2.0×10^{-2}	7.0×10^{-2}	3.5
3	7.4×10^{-3}	3.1×10^{-2}	4.2
10	4.2×10^{-4}	5.8×10^{-3}	13.7
50	2.5×10^{-10}	5.5×10^{-3}	2.2×10^{7}

A weight that sees a gradient of 10^{-10} cannot move: with early stopping monitoring a validation score that never improves, training halts within a few epochs and the deep network never leaves its random initialisation — which is exactly the chance-level accuracy we saw at depth 50. This confirms the collapse is an *optimisation* pathology, not a capacity or over-fitting one. The standard fixes (residual connections, batch/layer normalisation, He initialisation) all exist precisely to keep this gradient alive; none are warranted here, because the shallow network already solves the task.

4 How Stable Is the Result Across Seeds?

A single train/test split with a single weight initialisation gives one number. Would we get the same accuracy if we reshuffled the split or re-initialised the network? Both depend on a random seed (`random_state`), which fixes the stratified 80/20 train+dev / test partition and the starting weights. To gauge stability we retrain the selected configuration (all 107 features, depth 1) under ten seeds, $(42, 100, 200, \ldots, 900)$, and record the test accuracy each time. The dataset itself — including the fixed "none" class of Section 2 — is held constant, so the spread reflects split-and-initialisation noise rather than which passages happen to be sampled.

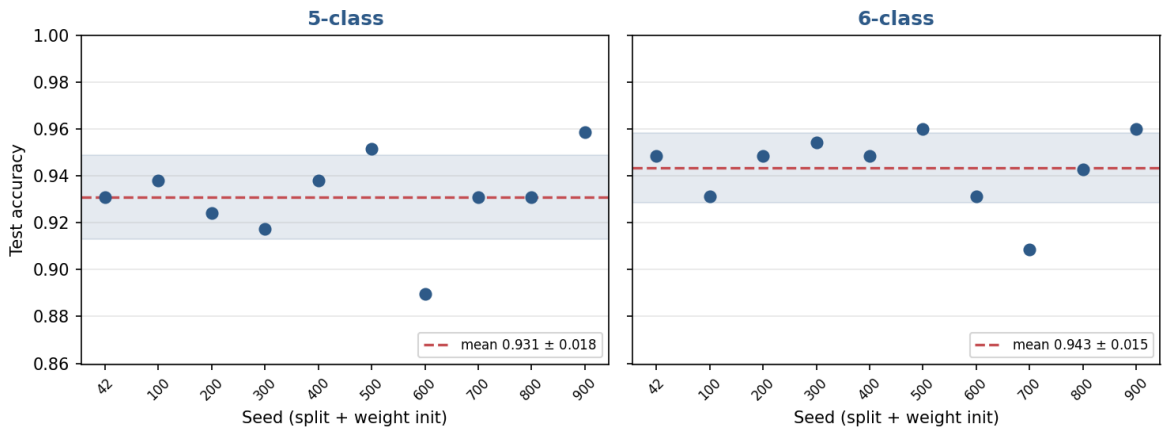


Figure 8: Test accuracy of the selected MLP across ten seeds. The dashed line is the mean; the shaded band is ± 1 standard deviation. Both tasks are tight: 0.931 ± 0.018 (5-class) and 0.943 ± 0.015 (6-class).

Across the ten seeds the 5-class model averages 0.931 test accuracy (standard deviation 0.018, range 0.890–0.959), and the 6-class model averages 0.943 (standard deviation 0.015, range 0.909–0.960). Two things are reassuring. First, the headline numbers we reported with `random_state=42` (0.931 and 0.949) sit right at or just above their respective means — they are representative runs, not lucky ones. Second, the 6-class spread is *tighter* than the 5-class spread despite the extra class, consistent with the perfectly-separable "none" class anchoring the decision boundaries. Even the worst seed in either sweep stays near 0.89, far above the random baselines, so the model's strong performance is a property of the features and architecture, not of one fortunate split.

5 Conclusion

A deliberately simple model — a single hidden layer of 64 ReLU units on 107 z-scored stylometric features — attributes LessWrong passages to their author with 93% accuracy across five authors and 95% across six, the sixth being an open-set "none of the above" class it recognises perfectly. Depth does not help and, past a point, destroys the model outright. The supervised picture is sharper than the unsupervised one: where K-means recovered ten-author structure at ARI ≈ 0.55, a labelled MLP separates a handful of authors almost cleanly, confirming that the stylistic signal is strong and largely linearly-accessible once the right features are in hand.

6 Setup and Reproducibility

All experiments run on CPU in well under a minute each. The code lives in two folders: `neural_network/` (5-class) and `neural_network_6class/` (open-set 6-class). The report figures are produced by `make_report_figures.py` and `gradient_diagnostics.py`.

Dependencies. Python ≥ 3.10 with a handful of scientific packages:

- `numpy`, `pandas` — data handling
- `scikit-learn` — `MLPClassifier`, splitting, scaling, metrics
- `matplotlib` — all figures
- `nltk` — tokenisation / POS-tagging when building the 6-class "none" class from raw articles (a regex fallback is used if NLTK data is unavailable)

Install. From the project root, ideally inside a virtual environment:

```
# from the project root; a virtual environment is recommended
python -m venv .venv
.venv\Scripts\activate          # Windows  (use: source .venv/bin/activate on macOS/Linux)
pip install numpy pandas scikit-learn matplotlib nltk
```

Run the models. Each script trains the full feature-subset × depth grid, selects on the dev split, retrains the winner, and writes `results.md` plus `roc.png` next to itself.

```
# 5-class MLP  ->  neural_network/results.md, roc.png
cd neural_network
python neural_network_code.py

# 6-class open-set MLP  ->  neural_network_6class/results.md, roc.png
cd ../neural_network_6class
python neural_network_6class.py
```

Regenerate the report figures. This reproduces most figures in this document — dev-selection heatmaps, the depth-effect plot, confusion matrices, ROC curves, and the seed-stability sweep — into `outputs/neural_network/`. The gradient-health diagnostic (Figure 7, Table 3) is a separate script.

```
cd neural_network
python make_report_figures.py      # heatmaps, depth, confusion, ROC, seed stability
python gradient_diagnostics.py     # per-layer gradient health -> gradient_health.png
```

Compile this report. Run `pdflatex` from the `docs/` folder (twice, so cross-references and the table of figures resolve):

```
cd docs
pdflatex neural_network_report.tex   # run twice to resolve cross-references
```